

Hackathon Submission (Level-1-Solution)

Use Case Title: Color Detection from Images

Student Name: Mohamed saif

Register Number: 712523205041

Institution: PPG Institute of technology

Department: BTECH IT

Date of Submission: 17/05/2025

1. Problem Statement

Designers, artists, and developers frequently need to identify precise colors from images for branding, content analysis, or accessibility. Manually determining the exact color value and its common name is time-consuming and error-prone. There is a need for a simple tool that allows users to upload an image, click anywhere on it, and instantly get the color name and RGB values at that pixel. This would streamline workflows, improve accuracy, and enhance digital design processes.

2. Proposed Solution

The proposed solution is a web-based Color Detection Application. Users can upload an image, click on any point, and immediately see both the RGB values and the closest matching color name from a predefined dataset. The application visually displays the detected color as a reference swatch. Key features include::

- User-friendly image upload and display.
- Real-time detection of RGB values upon clicking any pixel.
- Automatic matching of the detected color to a known color name using a CSV dataset.
- Visual display of both the color name, RGB values, and a color-filled rectangle.
- Responsive and clean user interface.
- Error handling for invalid files or missing datasets.

3. Technologies & Tools Considered

- ❖ Programming Language: Python
- ❖ Frameworks/Libraries: Streamlit (for UI), OpenCV (image processing), Pandas & NumPy (data handling and color matching)
- ❖ Dataset: colors.csv (predefined color names and RGB values)
- ❖ Version Control: GitHub (for code and documentation)

4. Solution Architecture & Workflow

- User Interface (Streamlit): Handles image upload, display, and user interactions.
- Image Processing (OpenCV): Loads the image, captures pixel data, and extracts RGB values.
- Data Matching (Pandas & NumPy): Loads the color dataset and finds the closest color name using Euclidean distance.
- Color Dataset (colors.csv): Provides the reference color names and RGB values.
- Workflow Steps:
 - User uploads an image via Streamlit.
 - Image is displayed; user clicks on any pixel.
 - App retrieves (x, y) coordinates and extracts RGB values.
 - RGB values are matched to the nearest color in the dataset.

- Color name, RGB values, and a visual swatch are displayed in real time..

5. Feasibility & Challenges

The solution is practical, leveraging widely-used Python libraries and Streamlit for rapid web deployment.

OpenCV and Pandas are well-documented and suitable for image and data processing tasks.

Hosting on GitHub with a clear README ensures easy setup and collaboration.

Challenges:

- Handling large image files or unsupported formats.
- Ensuring accurate color matching for similar shades.
- Managing missing or corrupted datasets.
- Providing a responsive UI for all device sizes.

Mitigation:

- Implement file validation and size checks.
- Use efficient color distance algorithms.
- Include fallback mechanisms for missing datasets.
- Design with responsive web principles.

6. Expected Outcome & Impact

- Functional Application: Allows users to upload images, click to detect colors, and view results instantly.
- Improved Workflow: Saves time and increases accuracy for designers, developers, and content creators.
- Accessibility: Assists in selecting accessible color palettes for digital content.
- Open Source: A well-documented GitHub repository benefits the wider community.

7. Future Enhancements

- Support for HEX and HSL color formats.
- Integration with online color palette generators.
- Batch processing for multiple color detections.
- Mobile-friendly or native mobile app versions.
- Accessibility features (e.g., color blindness simulation).
- Option to export detected colors and palettes.